BrandProof AI — Round 3 Final Submission

Trusted Content. Stronger Brands.

Summary

BrandProof AI checks brand content against an organization's approved facts, terminology and policies before it is published. A creator audits a draft, fixes it, sends it to a human reviewer, and an approved version receives a signed Content Passport. When an approved fact changes, only the content that depended on it is flagged for revalidation. Round 3 adds the three improvements promised in Round 2 and fixes the weaknesses we found in our own Round 2 build.

The three promised improvements

Promised improvement	Status	What it does
1. Gemini AI Content Auditor	Implemented	Gemini reviews tone, terminology, accessibility and policy, returning severity, a one-sentence reason and an evidence citation. Evidence IDs are checked against the organization's records, and the model cannot raise a high-severity finding without real evidence.
2. Smart Correction Engine	Implemented	Produces minimal edits, each with a reason and its source. Exact fixes come from the rules (approved fact values, approved terminology). Gemini rewrites are kept only if the rules confirm they cannot introduce a violation. The creator accepts or rejects each change.
3. Brand Knowledge Extraction	Implemented	Reads an uploaded guideline (PDF, text or Markdown) and proposes facts, terminology and policies. Every proposal carries a verbatim quote and page number. Nothing becomes evidence until an administrator approves it.

What changed from the Round 2 plan

- **Round 3 scope.** The Round 2 pack listed a different Round 3 roadmap (claim decomposition, signed provenance bundles, prioritized drift). We replaced it with the three improvements promised on our Round 2 slides. A lightweight

form of signed passports was added because it was small and strengthens the core promise.

- **Roles.** The Round 2 slides named five roles. The product implements three (Content Creator, Reviewer, Brand Administrator). Brand Manager and External Agency are not built, because scoped agency access needs real authentication. They are listed as future work.
- **Gemini model.** The default model is now the `gemini-flash-latest` alias rather than a pinned version, because pinned Gemini versions are retired over time. Settings includes a "Test AI connection" button that lists the models the key can use.
- **Rules stay authoritative.** Gemini adds suggestions and rewrites; it never overrides a rule finding, and every rewrite and extraction is verified by deterministic code.

How it works

1. **Rules first.** A deterministic engine checks every approved fact that contains a number and unit (for example 5000 mAh, 1 year, 6.1 inches, 65 W), prohibited claims, required phrases, superlatives and terminology. Facts an administrator adds are enforced automatically.

2. **Gemini second.** If a key is configured, Gemini receives only the retrieved organization context and the submitted text (marked as untrusted data). Its findings are schema-validated, de-duplicated against rule findings, and tagged as AI suggestions.

3. **Fallback.** If Gemini returns 429 or 5xx, the call is retried with backoff. If it still fails, the audit completes with rules only and the failure is shown on the audit record.

4. **Correct.** Rule fixes and filtered Gemini edits are presented as a per-change diff. The result is re-audited by the rules and shown as passing or not.

5. **Review and passport.** A human reviewer approves. The passport (content hash, fact and policy versions, findings, reviewer) is signed with HMAC-SHA256. A Verify action re-checks signature, content hash, evidence versions and approval state.

6. **Drift.** Changing or deactivating an approved fact or policy flags the approved content that relied on the older version.

Stack: Next.js and TypeScript frontend; FastAPI and SQLAlchemy backend; SQLite for the demo, PostgreSQL and Alembic supported; Gemini API over HTTPS with the key sent in a request header.

Round 2 problems fixed

- A creator could dismiss a high-severity "Contradicted" finding and get a wrong claim approved. High-severity findings now require a reviewer to resolve.
- The audit engine only knew three hardcoded NovaTech facts. It now enforces any numeric fact.
- Submitting without an audit, or with open findings, is blocked.
- Deactivating a fact or policy now triggers drift. Re-auditing no longer duplicates dependency records.
- An upgrade no longer wipes the database. Missing columns are added in place.
- The Gemini key is no longer placed in the request URL.
- Hardcoded names ("Good morning, Maya", "Demo Creator") were removed.
- Running the tests no longer touches the developer's own demo database.

Evaluation

We built a labelled set of 62 NovaTech claims (20 correct, 16 wrong fact, 14 unsupported claim, 12 exaggeration). Sixteen are "hard": paraphrases and claims the rules were not designed for. Results below were produced by `python eval/run_eval.py`.

Mode	Precision	Recall	False-positive rate	Recall on hard claims
Rules only	100.0%	61.9%	0.0%	0 of 16
Rules + Gemini	[TO FILL: run with `--gemini`]	[TO FILL]	[TO FILL]	[TO FILL]

Rules-only catches all 26 rules-style claims and misses all 16 hard ones. This is expected and is the gap Gemini is meant to close. The easy claims were written alongside the rules, so the rules-only precision is optimistic. Automatic rewrite check: all 26 flagged rules-style claims received a fix that re-audits clean. This is a proxy; whether people accept a rewrite is a human judgement we have not measured.

Testing notes

- The repository contains 62 automated test functions across five files: 29 from Round 2 and 33 added in Round 3.
- In the build environment where Round 3 was written, the 17 pure-logic tests (audit engine, correction engine, extraction, passport signing and an evaluation-set regression guard) were run and passed.
- The remaining tests need the backend dependencies and were written but not executed in that environment: [TO FILL: result of `cd backend && pytest -q`].
- The frontend was syntax-checked only: [TO FILL: result of `cd frontend && npm run build`].
- Gemini behaviour is covered by tests that mock the provider. Live output: [TO FILL after Settings > Test AI connection].

Known gaps

- Authentication is demo role headers, not real identity. Brand Manager and External Agency roles are not implemented.
- Visual checking is a local pixel comparison and optional OCR, not a brand-compliance model.
- Retrieval is lexical; pgvector search is not implemented.
- Passport signing uses a shared secret, so it proves integrity, not independent attestation.
- Background audits use an in-process worker, not a distributed queue.
- Gemini cannot be run without a key; rules-only mode is the default.

Team

[TO FILL: see `individual_contributions.md`.]